Atividade: Criar outro algoritmo

Introdução

Uma parte importante da cibersegurança é o controle de acesso a conteúdo restrito. Neste laboratório, você trabalhará com um arquivo de texto contendo endereços IP que têm permissão para acessar conteúdo restrito específico em sua organização.

A análise sintática de um arquivo permite que os analistas de segurança leiam e atualizem seu conteúdo. O Python auxilia os analistas no desenvolvimento de algoritmos para automatizar o processo de análise sintática de arquivos e mantê-los atualizados.

Você desenvolverá um algoritmo que analisa este arquivo de texto com endereços IP e o atualiza, removendo os endereços que não têm mais acesso ao conteúdo restrito.

Dicas para concluir este laboratório

Cenário

Neste laboratório, você trabalhará como analista de segurança e será responsável por desenvolver um algoritmo que analise um arquivo contendo endereços IP que têm permissão para acessar conteúdo restrito e remova os endereços que não têm mais acesso.

Tarefa 1

Seu objetivo final é desenvolver um algoritmo que analise uma série de endereços IP que podem acessar informações restritas e remova os endereços que não são mais permitidos. O Python pode automatizar esse processo.

Você recebe um arquivo de texto chamado `"allow_list.txt"` que contém uma série de endereços IP que têm permissão para acessar informações restritas.

Existem endereços IP que não devem mais ter acesso a essas informações e, portanto, precisam ser removidos do arquivo de texto. Você receberá uma variável chamada `ip_addresses` `remove_list` que contém a lista de endereços IP a serem removidos.

Exiba ambas as variáveis para explorar seus conteúdos e execute a célula. Certifique-se de substituir cada uma `### YOUR CODE HERE ###` pelo seu próprio código antes de executar a célula seguinte.

Em [1]:
```
# Atribua `import_file` ao nome do arquivo

import_file  =  "allow_list.txt"

# Atribua `remove_list` a uma lista de endereços IP que não têm mais permis
são para acessar informações restritas.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.
168.58.57"]

# Display `import_file`

print(import_file )

# Display `remove_list`

print(remove_list)
```

```
allow_list.txt
['192.168.97.225', '192.168.158.170', '192.168.201.40', '192.168.58.57']
```

Hint 1

Question 1

What do you observe about the output above?

Observa-se que import_file exibe apenas o nome do arquivo, enquanto remove_list exibe uma lista de endereços IP, mostrando que uma variável armazena uma string e a outra armazena uma lista.

Task 2

In this task, start by opening the text file using the `import_file` variable, the `with` keyword, and the `open()` function with the `"r"` parameter. Be sure to replace the `### YOUR CODE HERE ###` with your own code.

For now, you'll write the first line of the `with` statement. Running this code will produce an error because it will only contain the first line of the `with` statement; you'll complete this `with` statement in the task after this.

In [ ]:
```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed
to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.
168.58.57"]

# First line of `with` statement

with open(import_file, "r") as file:
```

Hint 1

Task 3

Now, use the `.read()` method to read the imported file and store it in a variable named `ip_addresses` .

Afterwards, display `ip_addresses` to examine the data in its current format.

Be sure to replace each `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [3]:
```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed
to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.
168.58.57"]

# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable name
d `ip_addresses`
      ip_addresses = file.read()


# Display `ip_addresses`

print(ip_addresses)
```

```
ip_address
192.168.25.60
192.168.205.12
192.168.97.225
192.168.6.9
192.168.52.90
192.168.158.170
192.168.90.124
192.168.186.176
192.168.133.188
192.168.203.198
192.168.201.40
192.168.218.219
192.168.52.37
192.168.156.224
192.168.60.153
192.168.58.57
192.168.69.116
```

Hint 1

Hint 2

Hint 3

Question 2

Do you notice any IP addresses in the allow list that are also in the `remove_list` ?

Sim, todos

Task 4

After reading the file, reassign the `ip_addresses` variable so its data type is updated from a string to a list. Use the `.split()` method to achieve this. Adding this step will allow you to iterate through each of the IP addresses in the allow list instead of navigating a large string that contains all the addresses merged together.

Afterwards, display the `ip_addresses` variable to verify that the update took place.

Be sure to replace each `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [4]:
```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed
to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.
168.58.57"]

# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable name
d `ip_addresses`

  ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()

# Display `ip_addresses`

print(ip_addresses)
```

```
['ip_address', '192.168.25.60', '192.168.205.12', '192.168.97.225', '192.1
68.6.9', '192.168.52.90', '192.168.158.170', '192.168.90.124', '192.168.18
6.176', '192.168.133.188', '192.168.203.198', '192.168.201.40', '192.168.2
18.219', '192.168.52.37', '192.168.156.224', '192.168.60.153', '192.168.5
8.57', '192.168.69.116']
```

Hint 1

Hint 2

Task 5

Now, you'll write code that removes the elements of `remove_list` from the `ip_addresses` list. This will require both an iterative statement and a conditional statement.

First, build the iterative statement. Name the loop variable `element`, loop through `ip_addresses`, and display each element. Be sure to replace each `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [8]:
```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed
to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.
168.58.57"]

# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable name
d `ip_addresses`

  ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

    # Display `element` in every iteration

    print(element)
```

```
ip_address
192.168.25.60
192.168.205.12
192.168.97.225
192.168.6.9
192.168.52.90
192.168.158.170
192.168.90.124
192.168.186.176
192.168.133.188
192.168.203.198
192.168.201.40
192.168.218.219
192.168.52.37
192.168.156.224
192.168.60.153
192.168.58.57
192.168.69.116
```

Hint 1

Hint 2

Task 6

Now, build a conditional statement to remove the elements of `remove_list` from the `ip_addresses` list. The conditional statement should be placed inside the iterative statement that loops through `ip_addresses`. In every iteration, if the current element in the `ip_addresses` list is in the `remove_list`, the `remove()` method should be used to remove that element.

Afterwards, display the updated `ip_addresses` list to verify that the elements of remove_list are no longer in the `ip_addresses`. Be sure to replace each `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [11]:
```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed
to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.
168.58.57"]

# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable name
d `ip_addresses`

  ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

  # Build conditional statement
  # If current element is in `remove_list`,

    if element in remove_list:

        # then current element should be removed from `ip_addresses`

        ip_addresses.remove(element)

# Display `ip_addresses`

print(ip_addresses)
```

```
['ip_address', '192.168.25.60', '192.168.205.12', '192.168.6.9', '192.168.
52.90', '192.168.90.124', '192.168.186.176', '192.168.133.188', '192.168.2
03.198', '192.168.218.219', '192.168.52.37', '192.168.156.224', '192.168.6
0.153', '192.168.69.116']
```

Hint 1

Hint 2

Hint 3

Task 7

The next step is to update the original file that was used to create the `ip_addresses` list. A line of code containing the `.join()` method has been added to the code so that the file can be updated. This is necessary because `ip_addresses` must be in string format when used inside the `with` statement to rewrite the file.

The `.join()` method takes in an iterable (such as a list) and concatenates every element of it into a string. The `.join()` method is applied to a string consisting of the character that will be used to separate every element in the iterable once its converted into a string. In the code below, the method is applied to the string `" "`, which contains just a space character. The argument of the `.join()` method is the iterable you want to convert, and in this case, that's `ip_addresses`. As a result, it converts `ip_addresses` from a list back into a string with a space between each element and the next.

After this line with the `.join()` method, build the `with` statement that rewrites the original file. Use the `"w"` parameter when calling the `open()` function to delete the contents in the original file and replace it with what you want to write. Be sure to replace each `### YOUR CODE HERE ###` with your own code before you run the following cell. This code cell will not produce an output.

In [14]:
```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed
to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.
168.58.57"]

# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable name
d `ip_addresses`

  ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

  # Build conditional statement
  # If current element is in `remove_list`,

    if element in remove_list:

        # then current element should be removed from `ip_addresses`

        ip_addresses.remove(element)

# Convert `ip_addresses` back to a string so that it can be written into th
e text file

ip_addresses = " ".join(ip_addresses)

# Build `with` statement to rewrite the original file

with open(import_file, "w") as file:

  # Rewrite the file, replacing its contents with `ip_addresses`

    file.write(ip_addresses)
```

Hint 1

Hint 2

Hint 3

Task 8

In this task, you'll verify that the original file was rewritten using the correct list.

Write another `with` statement, this time to read in the updated file. Start by opening the file. Then read the file and store its contents in the `text` variable.

Afterwards, display the `text` variable to examine the result.

Be sure to replace each `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [16]:
```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed
to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.
168.58.57"]

# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable name
d `ip_addresses`

  ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

  # Build conditional statement
  # If current element is in `remove_list`,

    if element in remove_list:

        # then current element should be removed from `ip_addresses`

        ip_addresses.remove(element)

# Convert `ip_addresses` back to a string so that it can be written into th
e text file

ip_addresses = " ".join(ip_addresses)

# Build `with` statement to rewrite the original file

with open(import_file, "w") as file:

  # Rewrite the file, replacing its contents with `ip_addresses`

  file.write(ip_addresses)

# Build `with` statement to read in the updated file

with open(import_file, "r") as file:

    # Read in the updated file and store the contents in `text`

    text = file.read()

# Display the contents of `text`
```

```
print(text)
```

```
ip_address 192.168.25.60 192.168.205.12 192.168.6.9 192.168.52.90 192.168.
90.124 192.168.186.176 192.168.133.188 192.168.203.198 192.168.218.219 19
2.168.52.37 192.168.156.224 192.168.60.153 192.168.69.116
```

Hint 1

Hint 2

Hint 3

Task 9

The next step is to bring all of the code you've written leading up to this point and put it all into one function.

Define a function named `update_file()` that takes in two parameters. The first parameter is the name of the text file that contains IP addresses (call this parameter `import_file`). The second parameter is a list that contains IP addresses to be removed (call this parameter `remove_list`).

Be sure to replace the `### YOUR CODE HERE ###` with your own code before you run the following cell. Note that this code cell will not produce an output.

In [19]:
```
# Define a function named `update_file` that takes in two parameters: `impo
rt_file` and `remove_list`
# and combines the steps you've written in this lab leading up to this

def update_file(import_file, remove_list):

    # Build `with` statement to read in the initial contents of the file

    with open(import_file, "r") as file:

        # Use `.read()` to read the imported file and store it in a variabl
e named `ip_addresses`

        ip_addresses = file.read()

    # Use `.split()` to convert `ip_addresses` from a string to a list

    ip_addresses = ip_addresses.split()

    # Build iterative statement
    # Name loop variable `element`
    # Loop through `ip_addresses`

    for element in ip_addresses:

        # Build conditional statement
        # If current element is in `remove_list`,

        if element in remove_list:

            # then current element should be removed from `ip_addresses`

            ip_addresses.remove(element)

    # Convert `ip_addresses` back to a string so that it can be written int
o the text file

    ip_addresses = " ".join(ip_addresses)

    # Build `with` statement to rewrite the original file

    with open(import_file, "w") as file:

        # Rewrite the file, replacing its contents with `ip_addresses`

        file.write(ip_addresses)
```

Hint 1

Hint 2

Hint 3

Question 3

What are the benefits of incorporating the algorithm into a single function?

Colocar tudo em uma única função deixa o código mais organizado, mais fácil de reutilizar e mais simples de manter. Se precisar mudar alguma coisa, você mexe em um lugar só, e ainda pode usar a mesma função quantas vezes quiser sem repetir código.

Task 10

Finally, call the `update_file()` that you defined. Apply the function to `"allow_list.txt"` and pass in a list of IP addresses as the second argument.

Use the following list of IP addresses as the second argument:

```
["192.168.25.60", "192.168.140.81", "192.168.203.198"]
```

After the function call, use a `with` statement to read the contents of the allow list. Then display the contents of the allow list. Run it to verify that the file has been updated by the function.

Be sure to replace the `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [20]:
```
# Define a function named `update_file` that takes in two parameters: `impo
rt_file` and `remove_list`
# and combines the steps you've written in this lab leading up to this

def update_file(import_file, remove_list):

  # Build `with` statement to read in the initial contents of the file

  with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable na
med `ip_addresses`

    ip_addresses = file.read()

  # Use `.split()` to convert `ip_addresses` from a string to a list

  ip_addresses = ip_addresses.split()

  # Build iterative statement
  # Name loop variable `element`
  # Loop through `ip_addresses`

  for element in ip_addresses:

    # Build conditional statement
    # If current element is in `remove_list`,

    if element in remove_list:

      # then current element should be removed from `ip_addresses`

      ip_addresses.remove(element)

  # Convert `ip_addresses` back to a string so that it can be written into
the text file

  ip_addresses = " ".join(ip_addresses)

  # Build `with` statement to rewrite the original file

  with open(import_file, "w") as file:

    # Rewrite the file, replacing its contents with `ip_addresses`

    file.write(ip_addresses)

# Call `update_file()` and pass in "allow_list.txt" and a list of IP addres
ses to be removed

update_file("allow_list.txt", ["192.168.25.60", "192.168.140.81", "192.168.
203.198"])

# Build `with` statement to read in the updated file

with open("allow_list.txt", "r") as file:

  # Read in the updated file and store the contents in `text`

  text = file.read()
```

```
# Display the contents of `text`

print(text)
```

```
ip_address 192.168.205.12 192.168.6.9 192.168.52.90 192.168.90.124 192.16
8.186.176 192.168.133.188 192.168.218.219 192.168.52.37 192.168.156.224 19
2.168.60.153 192.168.69.116
```

Hint 1

Hint 2

Hint 3

Conclusion

What are your key takeaways from this lab?

Python facilita muito a automação de tarefas de segurança, especialmente ao ler, analisar e atualizar arquivos de texto. Também fica claro como usar with, open(), .read(), .split(), .join() e funções ajuda a deixar o código mais organizado, reutilizável e confiável, algo essencial no dia a dia de cibersegurança.